KNOWLEDGE INSTITUTE OF TECHNOLOGY, SALEM
(AN AUTONOMOUS INSTITUTION)
Affiliated to Anna University, Chennai – 600 025

# SOLAR PANEL AUTO-CLEANING & EFFICIENCY MONITORING SYSTEM

CAPSTONE PROJECT REPORT

Submitted in partial fulfillment of the requirements for the award of the degree of

**BACHELOR OF ENGINEERING**

in

**COMPUTER SCIENCE AND ENGINEERING**

**Submitted by**

| Name | Registration Number |
|---|---|
| Barathkumar S (TL) | 611224104019 |
| Dharanidharan B | 611224104034 |
| Boobalan L | 611224104023 |
| Babu C P | 611224104017 |
| Arunbalaji S S | 611224104013 |
| Divyaprasath K R | 611224104047 |

Subject Faculty In-Charge: Mr. S. Sabarinathan, AP/ECE

OCTOBER 2026

# BONAFIDE CERTIFICATE

This is to certify that the Capstone Project Report entitled “Solar Panel Auto-Cleaning & Efficiency Monitoring System” are the bonafide work of Barathkumar S (TL) (Reg. No: 611224104019), Dharanidharan B (Reg. No: 611224104034), Boobalan L (Reg. No: 611224104023), Babu C P (Reg. No: 611224104017), Arunbalaji S S (Reg. No: 611224104013), and Divyaprasath K R (Reg. No: 611224104047), completed as part of the academic requirements.

SIGNATURE OF MENTOR

SIGNATURE OF HOD

**Mr. S. Sabarinathan**
ASSISTANT PROFESSOR
Department of Electronics and Communication Engineering
Knowledge Institute of Technology
Salem – 637504

**Mr. T. Karthikeyan**
HEAD OF THE DEPARTMENT
Department of Computer Science and Engineering
Knowledge Institute of Technology
Salem – 637504

Submitted for the Capstone Project Presentation held on _____.

INTERNAL EXAMINER

# TABLE OF CONTENTS


| | |
|---|---|
| Abstract and Keywords | 4 |
| Chapter 1: Introduction and Background | 4 |
| Chapter 2: Problem Statement | 4 |
| Chapter 3: Objectives | 4 |
| Chapter 4: Existing System and Limitations | 4 |
| Chapter 5: Proposed System and Advantages | 5 |
| Chapter 6: Hardware and Software Requirements | 5 |
| Chapter 7: Module Descriptions | 6 |
| Chapter 8: System Architecture and Data Flow | 6 |
| Chapter 9: Circuit Connections and Components | 7 |
| Chapter 10: Methodology and Working | 7 |
| Chapter 11: Source-Code Explanation | 8 |
| Chapter 12: Implementation Procedure | 8 |
| Chapter 13: Testing and Test Cases | 9 |
| Chapter 14: Results and Discussion | 9 |
| Chapter 15: Limitations | 9 |
| Chapter 16: Conclusion | 10 |
| Chapter 17: Future Scope | 10 |
| Chapter 18: References | 10–11 |
| Appendix A: Arduino Source Code | 11–13 |
| Appendix B: Prototype Evidence | 14 |

# ABSTRACT AND KEYWORDS

The Solar Panel Auto-Cleaning & Efficiency Monitoring System is an embedded control prototype intended to reduce the need for direct manual access when cleaning a solar panel. The implemented design uses an ESP32 microcontroller, its Wi-Fi access-point capability, an HTTP web server, and a servo-driven wiper mechanism. A phone or other Wi-Fi-capable device connects to the ESP32-hosted network and opens a simple web interface. The user can start the wiper, stop it, or command it to move to a home position.

The supplied Arduino sketch defines a servo control signal on GPIO 18 and sweeps the servo between 20° and 160° while the wiper is running. The current implementation does not measure dust, voltage, current, power, or conversion efficiency. Accordingly, this report describes remote cleaning control rather than claiming a measured efficiency improvement.

**Keywords:** ESP32, servo motor, solar panel cleaning, Wi-Fi access point, HTTP web server, embedded systems, remote control.

# CHAPTER 1: INTRODUCTION AND BACKGROUND

Photovoltaic panels convert sunlight into electrical energy. Dust and other deposits on the panel surface can obstruct incoming light and may reduce electrical output. Periodic cleaning is therefore part of maintaining a solar installation, but physically reaching rooftop or otherwise inaccessible panels can be inconvenient and can introduce safety risks.

This project demonstrates a compact, low-cost approach to operating a cleaning mechanism remotely. An ESP32 hosts a Wi-Fi network and a small web interface. Commands from the interface are handled by the ESP32, which drives a servo motor attached to a wiper mechanism. The scope of the implemented prototype is actuation and remote control; sensing and performance monitoring are identified as future work.

# CHAPTER 2: PROBLEM STATEMENT

Manual solar-panel cleaning requires a person to reach the panel and perform repeated cleaning. For rooftop installations, access may be difficult and climbing can be unsafe. Manual work also consumes time and may require repeated maintenance. The project addresses the need for a simple prototype that lets a user operate a panel-cleaning mechanism remotely, without physically reaching the panel for each operation.

# CHAPTER 3: OBJECTIVES

- Develop a low-cost solar-panel cleaning prototype.
- Use an ESP32 as the controller for the cleaning mechanism.
- Provide remote START, STOP, and HOME commands through a phone-accessible web interface.
- Use a servo motor to move a wiper across a solar-panel model.
- Reduce the need for direct physical access during operation.
- Establish a platform that can later be extended with dust sensing and electrical monitoring.

# CHAPTER 4: EXISTING SYSTEM AND LIMITATIONS

Common approaches include cleaning by hand with a cloth or brush, washing with water, or cleaning at fixed intervals. The supplied capstone report notes that some automated systems can be costly or complex for small installations.

### Limitations of the existing approaches

• Manual cleaning requires physical effort and direct access.

• Rooftop access can expose the person to falls or other hazards.

• Water-based cleaning requires water and additional manual work.

• Fixed-interval cleaning may occur when cleaning is not needed.

• More elaborate automatic systems may be unsuitable for small, low-cost setups.

# CHAPTER 5: PROPOSED SYSTEM AND ADVANTAGES

The proposed prototype combines an ESP32, a servo motor, and a simple wiper mechanism. The ESP32 operates as a Wi-Fi access point named **Solar_Wiper** and serves a mobile-friendly HTML page. The page provides START WIPER, STOP WIPER, and HOME buttons. The ESP32 receives the corresponding HTTP requests and changes the servo-control state.

### Advantages

• Remote operation from a phone or laptop connected to the ESP32 access point.

• A straightforward embedded architecture with one controller and one actuator.

• Reduced need for the operator to reach the panel during a cleaning cycle.

• A practical base for later sensing, monitoring, and automation.

### Scope clarification

The project title includes “Efficiency Monitoring,” but the uploaded code contains no voltage/current sensor, dust sensor, or efficiency calculation. Efficiency monitoring is not implemented in this version, and no quantitative efficiency gain is claimed.

# CHAPTER 6: HARDWARE AND SOFTWARE REQUIREMENTS

| Category | Item | Role / verified detail |
|---|---|---|
| Controller | ESP32 | Runs the Arduino program; creates Wi-Fi AP |
| Actuator | Servo motor | Wiper motion; control pin configured as GPIO 18 |
| Mechanical assembly | Wiper / cleaning mechanism | Moves across the panel model |
| Target | Solar panel or model | Surface used for the demonstration |
| Connectivity | Phone or laptop with Wi-Fi | Connects to the ESP32-hosted interface |
| IDE | Arduino IDE | Compile and upload the sketch |
| Libraries | WiFi.h, WebServer.h, ESP32Servo.h | Wi-Fi, HTTP server, and servo control |
| Language | Arduino C/C++ | Firmware implementation |

Power supply details, the exact ESP32 board variant, servo model, current rating, and full physical wiring are not specified in the source files reviewed for this report. Confirm these against the actual hardware before assembly or submission.

# CHAPTER 7: MODULE DESCRIPTIONS

## 7.1 User Interface Module

The program serves an HTML page titled “Solar Panel Wiper.” Its three controls navigate to /start, /stop, and /home. CSS styles the page for a phone-sized screen.

## 7.2 Wi-Fi Communication Module

The ESP32 is configured with WiFi.mode(WIFI_AP) and WiFi.softAP(). The network SSID in the sketch is Solar_Wiper. The code contains a literal password, which is reproduced in the source appendix for traceability; change it before public deployment if it is intended to protect access.

## 7.3 HTTP Server Module

A WebServer instance listens on port 80. The root route serves the page. The /start, /stop, and /home routes invoke their corresponding handlers and then redirect the browser to the root page with HTTP status 303.

## 7.4 Processing and Servo-Control Module

The Boolean variable wiperRunning controls whether the movement routine runs. The angle is incremented or decremented by one degree per eligible update. At 160°, direction reverses; at 20°, it reverses again.

## 7.5 Mechanical Cleaning Module

The servo drives the attached wiper. The actual mechanical linkage, contact pressure, and cleaning effectiveness depend on the physical prototype and are not quantified by the firmware.

# CHAPTER 8: SYSTEM ARCHITECTURE AND DATA FLOW

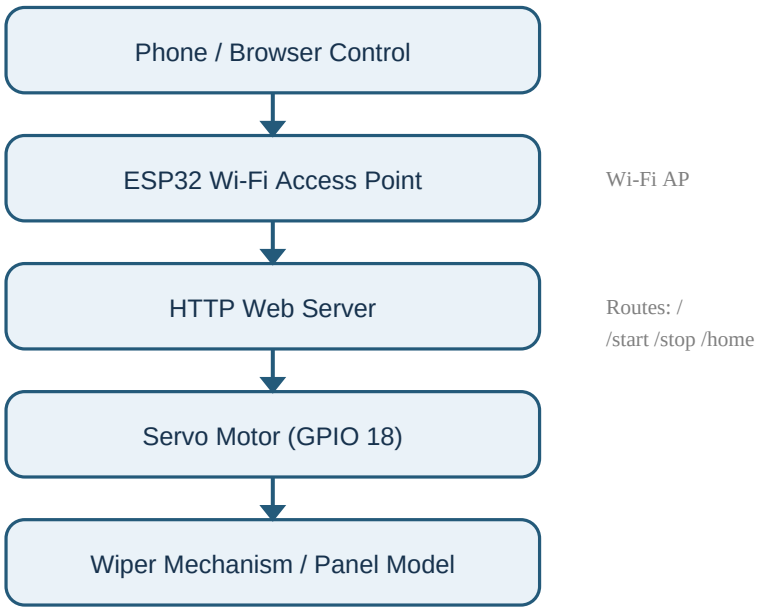


Figure 1. Functional architecture derived from the supplied Arduino sketch.

The command path is: phone/browser → ESP32 Wi-Fi access point → HTTP route handler → control state → servo output → wiper mechanism. The sketch does not use a cloud service or external router as part of this access-point setup.

### 8.1 Main software flow

• setup(): initializes Serial, configures and attaches the servo, starts the Wi-Fi access point, registers HTTP routes, and starts the server.

• loop(): services HTTP clients and, when enabled, updates the servo angle using millis() timing.

• handleStart(): sets wiperRunning to true.

• handleStop(): sets wiperRunning to false.

• handleHome(): stops motion, sets the software angle to 90°, resets direction, and commands the servo to 90°.

# CHAPTER 9: CIRCUIT CONNECTIONS AND COMPONENTS

| Connection / component | What can be verified | What remains unverified |
|---|---|---|
| Servo signal | Sketch defines SERVO_PIN = 18 | Physical wiring should be checked |
| Servo timing | 50 Hz; attach pulse limits 500–2400 µs | Servo model compatibility |
| Servo travel | Software limits 20° and 160°; HOME command 90° | Actual mechanical travel and linkage |
| ESP32 power | Controller is powered in the prototype | Exact supply and board variant |
| Servo power | Servo requires an appropriate supply | Voltage/current ratings and source |
| Grounding | Must follow the actual circuit design | Physical common-ground wiring |

Figure 2 is intentionally represented as a functional block diagram rather than a fabricated wiring schematic. The source code confirms a GPIO configuration, but it cannot prove the physical connection. Do not treat GPIO 18 as verified physical wiring until checked against the prototype.

### Electrical and mechanical checks

• Check the servo’s rated voltage and current before powering it.

• Do not power a servo directly from an ESP32 GPIO.

• If using a separate servo supply, follow the manufacturer’s grounding requirements.

• Test movement with the wiper clear of obstructions and mechanical stops.

# CHAPTER 10: METHODOLOGY AND WORKING

### 10.1 Startup

The ESP32 initializes serial communication at 115200 baud, configures the servo at 50 Hz, attaches it to GPIO 18, and commands the initial angle of 20°. It then starts a Wi-Fi access point and an HTTP server on port 80.

### 10.2 Remote operation

• Connect the phone or laptop to the Wi-Fi network named Solar_Wiper.

• Use the IP address printed by the ESP32 in Serial Monitor to open the web interface. The sketch prints WiFi.softAPIP(); do not assume an address if it has not been read from the device.

• Select START WIPER. The /start handler enables movement.

• Select STOP WIPER to disable movement.

• Select HOME to stop movement and command the servo to 90°.

### 10.3 Motion algorithm

While wiperRunning is true, loop() compares millis() with previousMillis. When the elapsed time reaches wiperSpeed (20 ms), the program updates the timestamp, changes the angle by the current direction, clamps at 20° or 160°, reverses direction at the limits, and writes the new angle to the servo.

# CHAPTER 11: SOURCE-CODE EXPLANATION

### 11.1 Libraries and server

WiFi.h provides ESP32 Wi-Fi functionality. WebServer.h supplies the HTTP server. ESP32Servo.h provides servo control. WebServer server(80) creates the server on port 80.

### 11.2 Configuration values

• SSID: Solar_Wiper.

• Password: 12345678 (as present in the uploaded sketch; change before public deployment if appropriate).

• SERVO_PIN: 18.

• LEFT_ANGLE / RIGHT_ANGLE: 20° / 160°.

• wiperSpeed: 20 milliseconds between eligible angle updates; this is a timing interval, not a calibrated physical speed.

### 11.3 State and route handlers

wiperRunning is the on/off state. angle tracks the commanded angle and direction indicates whether the angle is increasing or decreasing. The route handlers set this state or command the HOME position. Each handler redirects the browser to the root page using a 303 response.

### 11.4 Timing behavior and implementation note

The motion update is non-blocking with respect to the main loop because it uses millis() rather than delay(). The loop continues to call server.handleClient(). The servo is commanded one degree per 20 ms update, subject to loop timing and the servo's physical response. The code does not measure the actual shaft position.

### 11.5 Security note

The sketch embeds the access-point password in source code and prints it to Serial Monitor. For a public repository or deployed device, use a non-default password and avoid publishing credentials that should remain private.

# CHAPTER 12: IMPLEMENTATION PROCEDURE

### 12.1 Software setup

• Install Arduino IDE and ESP32 board support.

• Open sketch_oct3a.ino.

• Ensure the ESP32Servo library is installed and the Wi-Fi/WebServer headers are available in the selected ESP32 board package.

• Select the correct ESP32 board and serial port.

• Compile using Verify; address any compiler or library errors.

### 12.2 Hardware and deployment

• Connect the servo signal to the configured GPIO only after confirming the physical pinout.

• Provide suitable power to the ESP32 and servo according to their ratings.

• Attach the wiper mechanism and check that it moves freely through the commanded range.

• Upload the firmware.

• Open Serial Monitor at 115200 baud and note the printed access-point IP address.

• Connect a phone to Solar_Wiper, open the printed IP address in a browser, and test START, STOP, and HOME.

These are implementation steps based on the code and report. This document-generation process did not flash the ESP32 or perform a new physical test.

# CHAPTER 13: TESTING AND TEST CASES

The following test outcomes are transcribed from the supplied capstone report. They are reported project results, not tests independently repeated while preparing this document.

| Test case | Expected result | Status in supplied report |
|---|---|---|
| ESP32 operation | ESP32 operates correctly | Pass |
| Remote ON command | Servo starts cleaning movement | Pass |
| Remote OFF command | Servo stops cleaning movement | Pass |
| Servo response | Servo responds to ESP32 command | Pass |
| Cleaning mechanism | Wiper moves across panel | Pass |

## 13.1 Additional verification recommended

• Confirm that the phone can join the ESP32 access point and load the root page.

• Confirm each route (/start, /stop, /home) behaves as intended.

• Check that STOP halts commanded movement and HOME commands 90°.

• Check the full mechanical range for binding, collision, or excessive load.

• Repeat operation while monitoring power stability and servo temperature.

No quantitative duration, dust-removal percentage, electrical output, efficiency gain, or reliability rate is provided in the supplied report.

# CHAPTER 14: RESULTS AND DISCUSSION

The supplied report states that the prototype demonstrates remote-controlled cleaning using an ESP32 and servo motor. The user can send ON/OFF commands from a phone, and the servo-driven wiper moves across the panel model. The source code additionally implements a HOME command that commands the servo to 90°.

The demonstrated outcome is functional remote actuation. Since no dust sensor, voltage/current measurement, or efficiency calculation is implemented, the project does not establish a numerical improvement in panel output or efficiency. Such a claim would require measurements under documented and comparable conditions before and after cleaning.

# CHAPTER 15: LIMITATIONS OF THE CURRENT PROTOTYPE

• No dust sensor or dust-threshold detection.

• No voltage, current, power, or efficiency measurement.

• No automatic dust-triggered cleaning; operation is user-commanded.

• No cloud dashboard, mobile application backend, or remote Internet service is implemented in the sketch.

• No scheduled cleaning, RTC, rain sensor, or weather-aware control.

• Physical wiring, servo model, power ratings, and mechanical dimensions are not documented in the source materials reviewed.

• The code provides no authentication beyond the Wi-Fi access-point password and has no application-level user management.

• The reported functional tests do not provide quantitative cleaning or energy-performance measurements.

# CHAPTER 16: CONCLUSION

The team's capstone report describes a low-cost prototype for remote solar-panel cleaning. The uploaded firmware confirms an ESP32 access-point web server, START/STOP/HOME routes, and servo movement between configured limits. This design can reduce the need for the operator to physically reach the panel to initiate or stop the mechanism.

The present version should be understood as a remote-control cleaning prototype, not a completed efficiency-monitoring system. Its value is as a foundation for future sensing, measurement, and automation.

# CHAPTER 17: FUTURE SCOPE

• Add a dust sensor and define a calibrated threshold for cleaning decisions.

• Add a suitable voltage/current sensor, such as an appropriately selected INA219-based measurement arrangement.

• Record voltage, current, and power with timestamps and operating conditions.

• Develop a defensible before/after performance comparison and efficiency calculation.

• Add RTC-based scheduled operation.

• Consider rain or weather sensing and inhibit cleaning when conditions are unsuitable.

• Add a cloud dashboard or dedicated mobile interface if remote Internet access is required.

• Extend the system to multiple panels.

• Explore predictive maintenance only after collecting adequate, representative data.

• Add mechanical limit protection, obstruction handling, and safe behavior after communication loss.

# CHAPTER 18: REFERENCES

References below are reproduced from the supplied capstone report. They have not been independently bibliographically verified for this generated document.

• A. B. Y. Zaki and M. F. L. Abdullah, "Self-Sustaining Solar Panel Cleaning Solution with Integrated IoT Monitoring," J. Electron. Voltage Appl., vol. 6, no. 2, pp. 85–94, 2025. (As listed in the supplied capstone report.)

• K. C. Mane et al., "Automatic Cleaning and Maintenance of Solar Panel and Efficiency Monitoring," in Proc. 2nd Int. Conf. Edge Computing and Applications (ICECAA), Namakkal, India, 2023, pp. 1582–1586. (As listed in the supplied capstone report.)

• P. A. Patil, J. S. Bagi, and M. M. Wagh, "A review on cleaning mechanism of solar photovoltaic panel," in Proc. ICECDS, Chennai, India, 2017, pp. 250–256. (As listed in the supplied capstone report.)

• H. Abuzaid, M. Awad, and A. Shamayleh, “Impact of dust accumulation on photovoltaic panels: a review paper,” Int. J. Sustainable Engineering, vol. 15, no. 1, pp. 264–285, 2022. (As listed in the supplied capstone report.)

• S. A. Sulaiman, M. N. H. Mat, F. M. Guangul, and M. A. Bou-Rabee, “Real-time study on the effect of dust accumulation on performance of solar PV panels in Malaysia,” in Proc. ICEIT, 2015. (As listed in the supplied capstone report.)

# APPENDIX A: ARDUINO SOURCE CODE

The following listing is a normalized, line-wrapped reproduction of the uploaded sketch for readability. The authoritative source file is sketch_oct3a.ino. Keep that file with the report and repository.

```
#include <WiFi.h>
#include <WebServer.h>
#include <ESP32Servo.h>

const char* ssid = "Solar_Wiper";
const char* password = "12345678";

WebServer server(80);
Servo wiper;

const int SERVO_PIN = 18;
const int LEFT_ANGLE = 20;
const int RIGHT_ANGLE = 160;
int angle = 20;
int direction = 1;
bool wiperRunning = false;
unsigned long previousMillis = 0;
const int wiperSpeed = 20;

const char webpage[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Solar Panel Wiper</title>
<style>
body { font-family: Arial; text-align: center; background-color: #111;
       color: white; padding-top: 40px; }
h1 { color: #ff3333; font-size: 32px; }
p { font-size: 18px; }
button { width: 240px; height: 65px; margin: 12px; border: none;
         border-radius: 12px; font-size: 21px; font-weight: bold; }
.start { background-color: #00c853; color: white; }
.stop { background-color: #ff1744; color: white; }
.home { background-color: white; color: black; }
</style>
</head>
<body>
<h1>■ SOLAR PANEL WIPER</h1>
<p>Automatic Solar Panel Cleaning System</p>
<button class="start" onclick="location.href='/start'">START WIPER</button>
<br>
<button class="stop" onclick="location.href='/stop'">STOP WIPER</button>
<br>
<button class="home" onclick="location.href='/home'">HOME</button>
</body>
</html>
)rawliteral";
```

```
void handleRoot() {
    server.send(200, "text/html", webpage);
}

void handleStart() {
    wiperRunning = true;
    server.sendHeader("Location", "/");
    server.send(303);
    Serial.println("WIPER STARTED");
}

void handleStop() {
    wiperRunning = false;
    server.sendHeader("Location", "/");
    server.send(303);
    Serial.println("WIPER STOPPED");
}

void handleHome() {
    wiperRunning = false;
    angle = 90;
    direction = 1;
    wiper.write(90);
    server.sendHeader("Location", "/");
    server.send(303);
    Serial.println("WIPER HOME");
}

void setup() {
    Serial.begin(115200);
    wiper.setPeriodHertz(50);
    wiper.attach(SERVO_PIN, 500, 2400);
    angle = LEFT_ANGLE;
    wiper.write(angle);

    WiFi.mode(WIFI_AP);
    WiFi.softAP(ssid, password);

    Serial.println();
    Serial.println("==============================");
    Serial.println("SOLAR PANEL WIPER");
    Serial.println("==============================");
    Serial.print("WiFi Name: ");
    Serial.println(ssid);
    Serial.print("Password: ");
    Serial.println(password);
    Serial.print("IP Address: ");
```

```
    Serial.println(WiFi.softAPIP());

    server.on("/", handleRoot);
    server.on("/start", handleStart);
    server.on("/stop", handleStop);
    server.on("/home", handleHome);
    server.begin();
    Serial.println("Web server started");
}

void loop() {
    server.handleClient();

    if (wiperRunning) {
        unsigned long currentMillis = millis();
        if (currentMillis - previousMillis >= wiperSpeed) {
            previousMillis = currentMillis;
            angle = angle + direction;

            if (angle >= RIGHT_ANGLE) {
                angle = RIGHT_ANGLE;
                direction = -1;
            }
            if (angle <= LEFT_ANGLE) {
                angle = LEFT_ANGLE;
                direction = 1;
            }
            wiper.write(angle);
        }
    }
}
```

# APPENDIX B: PROTOTYPE EVIDENCE

The supplied capstone report includes a final-prototype figure. The original report PDF should be retained as the source of that photograph. A separate prototype photograph was not confidently identifiable among the available image files, so no unrelated image has been presented as the final hardware prototype.

Suggested evidence to include in the final submission: (1) a clear overall photograph of the assembled panel, wiper, servo, and ESP32; (2) a close-up of the ESP32/servo wiring; and (3) a genuine phone screenshot showing the START WIPER, STOP WIPER, and HOME interface. Label each image with its actual capture context.

**Figure B.1.** Prototype photograph: refer to Appendix B, Figure A.2 in the supplied original capstone report PDF.

**Figure B.2.** Mobile web interface: screenshot not supplied as a separate verified image in the materials used for this report.